

Dryad Programming Language

Fundamentos Teóricos e Especificação Formal

Theoretical Foundation Document

Version 1.0

May 27, 2026

Contents

1	Introdução e Filosofia da Linguagem	5
1.1	Visão Geral	5
1.2	Características Distintivas	5
1.3	Princípios de Design	6
2	Estrutura Lexical	6
2.1	Alfabeto e Caracteres	6
2.2	Tokens e Classificação	6
2.3	Palavras Reservadas	7
2.4	Operadores	8
2.5	Comentários e Whitespace	9
2.6	Diretivas Nativas e Desambiguação Lexical	9
3	Sistema de Tipos	10
3.1	Fundamentos do Sistema de Tipos	10
3.2	Tipos Primitivos	10
3.3	Tipos Compostos	10
3.4	Anotações de Tipo	11
3.5	Modo Estrito de Tipos (Futuro)	12
3.6	Coerção e Compatibilidade de Tipos	13
4	Expressões	13
4.1	Hierarquia de Precedência	13
4.2	Expressões Primárias	14
4.3	Expressões Binárias	15
4.4	Expressões Unárias	15
4.5	Acesso a Propriedades e Chamadas	16
4.6	Match Expression	16
5	Statements (Instruções)	16
5.1	Declarações de Variáveis	16
5.2	Statements de Controle de Fluxo	17
5.3	Tratamento de Exceções	18
5.4	Return Statement	18
6	Declarações	19
6.1	Funções	19
6.2	Classes	20
7	Sistema de Módulos	21
7.1	Import Statement	21
7.2	Export Statement	22
7.3	Native Directives	22

8	Modelo de Execução	22
8.1	Arquitetura de Execução Híbrida	22
8.2	Pipeline de Compilação	23
8.3	Escopo e Resolução de Variáveis	23
8.4	Execução de Funções	24
8.5	Instanciação de Classes	25
8.6	Representação Interna de Valores	25
8.7	Gerenciamento de Memória	26
9	Sistema de Erros	26
9.1	Categorias de Erros	26
9.2	Propagação de Exceções	27
10	Modelo de Concorrência	28
10.1	Async/Await	28
10.2	Threads Nativos	28
10.3	Sincronização com Mutex	28
11	Foreign Function Interface (FFI)	29
11.1	Sistema de Módulos Nativos	29
11.2	Mapeamento de Tipos	29
11.3	Convenção de Chamada	30
12	Arquitetura de Runtime Mínimo com Intrinsics	30
12.1	Filosofia do Self-Hosting	30
12.2	Sistema de Intrinsics	30
12.3	Pipeline de Compilação com Intrinsics	31
12.4	Implementação no Runtime C++	31
12.5	Standard Library em Dryad Puro	32
12.6	Virtual File System (VFS)	33
12.7	Networking e Async I/O	34
12.8	Gerenciamento de Memória com Buffer	36
12.9	Conjunto Mínimo de Syscalls	37
12.10	Comparação com Sistema Anterior	38
13	Bytecode e IR	38
13.1	Bytecode VM	38
13.2	Intermediate Representation (IR)	39
14	Compilação AOT (Ahead-of-Time)	39
14.1	Pipeline AOT	39
14.2	Backends Suportados	40
14.3	Formatos de Objeto	40
15	Padrões de Programação	40
15.1	Paradigma Orientado a Objetos	40
15.2	Paradigma Funcional	41
15.3	Paradigma Procedural	41

16 Especificações Formais	41
16.1 Gramática Formal (BNF Simplificada)	41
16.2 Semântica Operacional (Informal)	42
17 Limitações e Considerações	43
17.1 Limitações Conhecidas	43
17.2 Características de Performance	43
18 Conclusão	43
18.1 Resumo	43
18.2 Filosofia de Design	44
18.3 Direções Futuras	44

Abstract

Este documento apresenta os fundamentos teóricos completos da linguagem de programação Dryad, uma linguagem multi-paradigma de tipagem dinâmica inspirada em JavaScript/TypeScript, mas com características próprias que a distinguem. A especificação cobre aspectos lexicais, sintáticos, semânticos, sistema de tipos, modelo de execução, sistema de módulos, concorrência, e o modelo de compilação híbrido (interpretador + bytecode + AOT). Este documento é puramente teórico e não contém implementações de código, servindo como referência fundamental para compreensão conceitual da linguagem e base para futuras reimplementações.

1 Introdução e Filosofia da Linguagem

1.1 Visão Geral

Dryad é uma linguagem de programação multi-paradigma projetada com os seguintes objetivos fundamentais:

- **Familiaridade Sintática:** Sintaxe inspirada em JavaScript/TypeScript para reduzir curva de aprendizado
- **Flexibilidade de Paradigma:** Suporte simultâneo para programação procedural, orientada a objetos e funcional
- **Tipagem Dinâmica Opcional:** Sistema de tipos dinâmico com anotações opcionais para documentação e verificação
- **Modelo de Execução Híbrido:** Suporte para interpretação direta de AST, compilação para bytecode e compilação AOT para código nativo
- **Concorrência Nativa:** Primitivas de concorrência integradas (threads, async/await, mutexes)
- **Sistema de Módulos Robusto:** Import/export moderno com suporte a módulos nativos via FFI
- **Gerenciamento Automático de Memória:** Garbage collection transparente

1.2 Características Distintivas

Definição 1.1 (Características Fundamentais). Dryad se diferencia por:

1. **Arquitetura de Execução Tripla:** Interpretador de AST, VM de bytecode stack-based, e compilador AOT
2. **Sistema de Erros Estruturado:** Todos os erros são categorizados com códigos únicos, localizações e sugestões
3. **Operador de Namespace:** Suporte ao operador `::` para acesso a namespaces (estilo C++/Rust)
4. **Native Directives:** Carregamento explícito de módulos nativos via diretivas `#<module>`
5. **Template Strings:** Interpolação de strings nativa com `'${expr}'`

1.3 Princípios de Design

Axioma 1.1 (Princípios Fundamentais). A linguagem Dryad é regida pelos seguintes princípios:

- **Clareza sobre Performance:** Prioriza-se legibilidade e clareza do código sobre otimização prematura
- **Consistência Sintática:** Estruturas sintáticas seguem padrões consistentes
- **Erros Informativos:** Todo erro deve fornecer localização, contexto e sugestões de correção
- **Composabilidade:** Primitivas da linguagem devem ser combináveis
- **Zero Surpresas:** Comportamento da linguagem deve ser previsível e documentado

2 Estrutura Lexical

2.1 Alfabeto e Caracteres

Definição 2.1 (Alfabeto Dryad). O alfabeto da linguagem Dryad é composto por:

- Caracteres Unicode UTF-8
- Letras: [a-zA-Z]
- Dígitos: [0-9]
- Símbolos especiais: { } () [] ; , . : = + - * / % & | ^ ! < > ? @ # \$ \ ' " ‘
- Whitespace: espaços, tabulações, quebras de linha

2.2 Tokens e Classificação

Definição 2.2 (Sistema de Tokens). A análise lexical de Dryad produz os seguintes tipos de tokens:

1. **Identificadores:** Identifier(String)
Padrão: [a-zA-Z_][a-zA-Z0-9_]*
2. **Literais Numéricos:** Number(f64)
Suporta: decimal, hexadecimal (0x), binário (0b), octal (0o)
3. **Literais String:** String(String)
Delimitadores: " ou '
4. **Literais Booleanos:** Boolean(bool)
Valores: true, false
5. **Literal Null:** Literal("null")

6. **Template Strings:**
 TemplateStart, TemplateContent(String),
 InterpolationStart, InterpolationEnd, TemplateEnd
7. **Palavras-chave:** Keyword(String)
8. **Operadores:** Operator(String)
9. **Símbolos:** Symbol(char)
10. **Diretivas Nativas:** NativeDirective(String)
 Formato: #<module_name>
11. **Fim de Arquivo:** Eof

2.3 Palavras Reservadas

Definição 2.3 (Conjunto de Keywords). As palavras-chave reservadas de Dryad são organizadas por categoria:

Declaração de Variáveis:

- `let` (variável mutável)
- `const` (variável imutável)

Controle de Fluxo:

- `if`, `else`
- `for`, `while`, `do`
- `break`, `continue`
- `match`, `in`

Funções:

- `function`, `fn`
- `async`, `await`
- `return`
- `thread`, `mutex`

Orientação a Objetos:

- `class`, `new`
- `extends`, `this`, `super`
- `static`, `public`, `private`, `protected`

Módulos:

- `import`, `export`
- `use`, `as`, `from`

Tratamento de Erros:

- `try`, `catch`, `finally`
- `throw`

Literais:

- `true`, `false`, `null`

2.4 Operadores

Definição 2.4 (Sistema de Operadores). Dryad possui os seguintes operadores organizados por categoria:

Operadores Aritméticos:

- + (adição/concatenação)
- - (subtração)
- * (multiplicação)
- / (divisão)
- % (módulo)
- ** (exponenciação)

Operadores Bitwise:

- & (AND bitwise)
- | (OR bitwise)
- ^, ^^ (XOR bitwise)
- « (shift left)
- » (shift right aritmético)
- >> (shift right unsigned)
- «< (rotate left)
- %% (módulo bitwise)

Operadores de Comparação:

- == (igualdade)
- != (desigualdade)
- < (menor que)
- > (maior que)
- <= (menor ou igual)
- >= (maior ou igual)

Operadores Lógicos:

- && (AND lógico)
- || (OR lógico)
- ! (NOT lógico)

Operadores de Atribuição:

- = (atribuição)
- +=, -=, *=, /= (atribuições compostas)

Operadores de Incremento/Decremento:

- ++ (incremento, prefix ou postfix)
- - (decremento, prefix ou postfix)

Operadores Especiais:

- => (arrow function)
- # (native directive prefix)
- :: (namespace access)

2.5 Comentários e Whitespace

Definição 2.5 (Comentários). Dryad suporta dois tipos de comentários:

- **Linha:** `// comentário até o fim da linha`
- **Bloco:** `/* comentário multi-linha */`

Comentários são descartados durante análise lexical e não fazem parte da AST.

Propriedade 2.1 (Whitespace). Espaços em branco (espaços, tabs, newlines) são geralmente insignificantes, exceto:

- Como separadores entre tokens
- Em strings literais
- Em template strings

Não há significado sintático para indentação (diferente de Python).

2.6 Diretivas Nativas e Desambiguação Lexical

Definição 2.6 (Diretiva Nativa). Uma diretiva nativa tem o formato:

`#<module_name>`

onde `module_name` é um identificador válido delimitado por `<` e `>`.

Propriedade 2.2 (Desambiguação do Símbolo `#`). O caractere `#` em Dryad possui dois contextos de uso:

1. **Diretiva Nativa:** Quando seguido imediatamente (sem whitespace) por `<`, é interpretado como início de diretiva nativa
2. **Operador Futuro:** Reservado para potenciais operadores futuros

O analisador lexical garante distinção clara através das seguintes regras:

- Se `#` é seguido de `<` (sem espaços): token `NativeDirective`
- Caso contrário: erro léxico (caractere inesperado)

Axioma 2.1 (Princípio da Não-Ambiguidade). O analisador lexical de Dryad garante que:

$\forall$ sequência de entrada s , $\exists!$ interpretação léxica t tal que $lex(s) = t$

Ou seja, para qualquer entrada válida existe uma única interpretação de tokens possível.

Exemplos de Análise:

<code>#<io></code>	$\rightarrow$ <code>NativeDirective("io")</code>
<code># <io></code>	$\rightarrow$ Erro Léxico (espaço após <code>#</code>)
<code>//#<io></code>	$\rightarrow$ Comentário (descartado)
<code>"#<io>"</code>	$\rightarrow$ String Literal

3 Sistema de Tipos

3.1 Fundamentos do Sistema de Tipos

Definição 3.1 (Tipagem Dinâmica). Dryad implementa um **sistema de tipos dinâmico** onde:

- Tipos são associados a valores, não a variáveis
- Variáveis podem mudar de tipo durante execução
- Verificação de tipo ocorre em tempo de execução
- Anotações de tipo são opcionais e servem para documentação

3.2 Tipos Primitivos

Definição 3.2 (Tipos Primitivos de Dryad). Os tipos primitivos fundamentais são:

1. **number**: Número de ponto flutuante (IEEE 754 f64)
Literais: `42`, `3.14`, `0xFF`, `1e10`
2. **string**: Sequência de caracteres UTF-8
Literais: `"hello"`, `'world'`, `'template'`
3. **bool**: Valor booleano
Literais: `true`, `false`
4. **null**: Valor nulo/indefinido
Literal: `null`
5. **any**: Tipo universal (implícito)
Pode conter qualquer valor

3.3 Tipos Compostos

Definição 3.3 (Arrays). Arrays são coleções heterogêneas ordenadas de tamanho dinâmico.

Sintaxe de Tipo: `Type[]`

Propriedades:

- Indexação baseada em zero
- Tamanho dinâmico
- Podem conter tipos mistos quando declarados como `any[]`
- Acesso por índice: `arr[i]`

Definição 3.4 (Tuples). Tuplas são coleções heterogêneas de tamanho fixo.

Sintaxe de Tipo: `(Type1, Type2, ...)`

Propriedades:

- Tamanho fixo definido na criação

- Cada posição tem tipo potencialmente diferente
- Acesso por índice numérico: `tuple.0`, `tuple.1`

Definição 3.5 (Functions). Funções são valores de primeira classe.

Sintaxe de Tipo: `fn(ParamType1, ParamType2, ...) -> ReturnType`

Propriedades:

- Podem ser atribuídas a variáveis
- Podem ser passadas como argumentos
- Podem ser retornadas de outras funções
- Suportam closures (capturam escopo)

Definição 3.6 (Objects). Objetos são coleções de pares chave-valor.

Propriedades:

- Chaves são strings
- Valores podem ser de qualquer tipo
- Acesso via notação ponto: `obj.prop`
- Acesso via notação colchetes: `obj["prop"]`

Definição 3.7 (Classes). Classes são tipos definidos pelo usuário com estado e comportamento.

Propriedades:

- Suportam herança via `extends`
- Possuem propriedades de instância e estáticas
- Possuem métodos de instância e estáticos
- Suportam modificadores de acesso: `public`, `private`, `protected`

3.4 Anotações de Tipo

Definição 3.8 (Sintaxe de Anotações). Anotações de tipo aparecem após identificadores com sintaxe :

Exemplos:

- Variáveis: `let x: number = 42`
- Parâmetros: `function add(a: number, b: number)`
- Retorno: `function add(...): number`

Regras:

- Se omitida, tipo é inferido ou `any`
- Anotações são verificadas em parse time mas não enforced em runtime
- Servem primariamente para documentação

3.5 Modo Estrito de Tipos (Futuro)

Definição 3.9 (Strict Type Mode). O **modo estrito** é uma extensão futura planejada onde anotações de tipo são validadas estaticamente pelo compilador AOT para gerar código otimizado.

Propriedade 3.1 (Benefícios do Modo Estrito). Quando habilitado, o modo estrito fornece:

- **Otimização AOT:** Compilador gera código de máquina especializado sem overhead de verificação dinâmica de tipos
- **Eliminação de Verificações Runtime:** Tipos conhecidos em tempo de compilação permitem eliminar type checks
- **Especialização de Código:** Funções genéricas podem ser monomorfizadas para tipos concretos
- **Deteção de Erros Antecipada:** Incompatibilidades de tipo detectadas antes da execução
- **Melhor Performance:** Redução de 30-50% no overhead de tipo em hot paths

Axioma 3.1 (Compatibilidade de Modo). O modo estrito deve ser **opt-in** e compatível com código dinâmico:

- Módulos podem especificar "use strict types" no topo do arquivo
- Código estrito pode interoperar com código dinâmico através de fronteiras explícitas
- Funções individuais podem ser marcadas como `@strict` ou `@dynamic`
- Conversão implícita na fronteira: tipos dinâmicos são verificados ao entrar em código estrito

Exemplo Conceitual (Futuro):

```
"use strict types";

// Função estrita: tipos verificados estaticamente
function add(a: number, b: number): number {
    return a + b; // Compilador garante que a + b é number
}

// Compilado para código de máquina sem type checks:
// mov rax, [a]
// add rax, [b]
// ret

// Função dinâmica (fallback)
@dynamic
function process(x: any): any {
    return x + 1; // Runtime type check necessário
}
```

Propriedade 3.2 (Otimizações Habilitadas). Com tipos estaticamente conhecidos, o compilador AOT pode aplicar:

1. **Inline Caching Eliminado:** Sem necessidade de cache polimórfico
2. **Devirtualização:** Chamadas de método resolvidas estaticamente
3. **Escape Analysis:** Alocações de stack substituem heap quando possível
4. **Dead Code Elimination:** Branches impossíveis removidas
5. **SIMD Vectorization:** Loops sobre arrays tipados vetorizados automaticamente

3.6 Coerção e Compatibilidade de Tipos

Propriedade 3.3 (Conversões Implícitas). Dryad realiza as seguintes conversões automáticas:

- `number + string` $\rightarrow$ `string` (concatenação)
- `bool` em contexto aritmético $\rightarrow$ 1 (true) ou 0 (false)
- `null` em comparações tem tratamento especial
- Arrays e objetos são comparados por referência

4 Expressões

4.1 Hierarquia de Precedência

Definição 4.1 (Tabela de Precedência). Expressões em Dryad seguem a seguinte hierarquia de precedência (do maior para o menor):

1. **Primárias:** `.`, `[]`, `()`, `new` (left-associative)
2. **Postfix:** `++`, `-` (left-associative)
3. **Prefix:** `++`, `-`, `!`, `-`, `+`, `~` (right-associative)
4. **Exponenciação:** `**` (right-associative)
5. **Multiplicativas:** `*`, `/`, `%` (left-associative)
6. **Aditivas:** `+`, `-` (left-associative)
7. **Bitwise Shift:** `«`, `»`, `»»`, `««` (left-associative)
8. **Relacionais:** `<`, `<=`, `>`, `>=`, `in` (left-associative)
9. **Igualdade:** `==`, `!=` (left-associative)
10. **Bitwise AND:** `&` (left-associative)
11. **Bitwise XOR:** `^`, `^^` (left-associative)

12. **Bitwise OR:** `|` (left-associative)
13. **Logical AND:** `&&` (left-associative)
14. **Logical OR:** `||` (left-associative)
15. **Atribuição:** `=, +=, -=, *=, /=` (right-associative)

4.2 Expressões Primárias

Definição 4.2 (Literais). Expressões literais representam valores constantes:

- Números: `42`, `3.14`
- Strings: `"text"`, `'text'`
- Booleanos: `true`, `false`
- Null: `null`
- Template strings: `'text ${expr}'`

Definição 4.3 (Arrays e Tuplas Literais). **Arrays:** `[elem1, elem2, ...]`

- Podem conter elementos de tipos diferentes
- Suportam spread operator: `[...arr]`

Tuplas: `(elem1, elem2, ...)`

- Tamanho fixo
- Tipos heterogêneos

Definição 4.4 (Objetos Literais). Sintaxe: `{ key1: value1, key2: value2, ... }`

Variações:

- Propriedades: `{ x: 1, y: 2 }`
- Métodos: `{ greet() { ... } }`
- Chaves computadas: `{ ["key"]: value }`

Definição 4.5 (Funções Lambda (Arrow Functions)). Sintaxe: `(params) => expression` ou `(params) => { statements }`

Formas:

- Sem parâmetros: `() => expr`
- Um parâmetro: `x => expr`
- Múltiplos: `(a, b) => expr`
- Com bloco: `(a, b) => { return a + b; }`
- Rest params: `(a, ...rest) => expr`

4.3 Expressões Binárias

Definição 4.6 (Operações Aritméticas). Semântica das operações aritméticas:

- $a + b$: Se ambos números, soma. Se algum é string, concatena.
- $a - b$: Subtração (requer números)
- $a * b$: Multiplicação (requer números)
- a / b : Divisão (requer números, erro se $b = 0$)
- $a \% b$: Módulo (requer números, erro se $b = 0$)
- $a ** b$: Exponenciação (requer números)

Definição 4.7 (Operações Lógicas). Operadores lógicos com short-circuit evaluation:

- $a \&\& b$: Retorna a se falsy, senão b
- $a || b$: Retorna a se truthy, senão b
- $!a$: Negação booleana

Valores Falsy: `false`, `null`, `0`, `""`, `NaN`

Valores Truthy: Todos os demais

4.4 Expressões Unárias

Definição 4.8 (Operadores Unários). • $-x$: Negação numérica

- $+x$: Plus unário (converte para número)
- $!x$: NOT lógico
- $\sim x$: NOT bitwise
- $++x$: Pré-incremento
- $--x$: Pré-decremento
- $x++$: Pós-incremento
- $x--$: Pós-decremento
- `await expr`: Espera resolução de promise

4.5 Acesso a Propriedades e Chamadas

Definição 4.9 (Acesso a Membros). • **Ponto:** `obj.property`

- **Colchetes:** `obj["property"], arr[index]`
- **Tupla:** `tuple.0, tuple.1`
- **Namespace:** `module::function`

Definição 4.10 (Chamadas de Função). Sintaxe: `function(arg1, arg2, ...)`

Variações:

- Chamada direta: `fn()`
- Chamada de método: `obj.method()`
- Chamada com new: `new Class()`

4.6 Match Expression

Definição 4.11 (Pattern Matching). Sintaxe:

```
match valor {
  pattern1 => resultado1,
  pattern2 => resultado2,
  _ => default
}
```

Patterns Suportados:

- Literais: `1, "text", true`
- Wildcard: `_`
- Guards: condições opcionais após `=>`

5 Statements (Instruções)

5.1 Declarações de Variáveis

Definição 5.1 (Let Statement). Declara variável mutável com escopo de bloco.

Sintaxe: `let identifier [: type] [= value];`

Propriedades:

- Escopo de bloco (limitado ao bloco `{}` mais próximo)
- Pode ser reatribuída
- Se não inicializada, valor é `undefined/null`
- Múltiplas declarações: `let a = 1, b = 2;`

Definição 5.2 (Const Statement). Declara variável imutável com escopo de bloco.

Sintaxe: `const identifier [: type] = value;`

Propriedades:

- Escopo de bloco
- Deve ser inicializada na declaração
- Não pode ser reatribuída
- Para objetos/arrays, o conteúdo pode ser modificado

5.2 Statements de Controle de Fluxo

Definição 5.3 (If Statement). Execução condicional.

Sintaxe:

```
if (condition) {
    // then block
} else if (condition2) {
    // else-if block
} else {
    // else block
}
```

Semântica:

- Condição é avaliada como truthy/falsy
- Apenas o primeiro bloco verdadeiro é executado
- `else` e `else if` são opcionais

Definição 5.4 (Loops). **While Loop:**

```
while (condition) {
    // body
}
```

Executa enquanto condição for truthy.

Do-While Loop:

```
do {
    // body
} while (condition);
```

Executa pelo menos uma vez, depois verifica condição.

For Loop (C-style):

```
for (init; condition; update) {
    // body
}
```

Estrutura clássica de iteração.

For-Each Loop:

```
for (element in iterable) {
    // body
}
```

Itera sobre elementos de coleção.

Definição 5.5 (Break e Continue). • **break**: Sai imediatamente do loop atual

- **continue**: Pula para próxima iteração do loop
- Ambos afetam apenas o loop mais interno.

5.3 Tratamento de Exceções

Definição 5.6 (Try-Catch-Finally). Sintaxe:

```
try {
    // código que pode lançar exceção
} catch (error) {
    // tratamento de erro
} finally {
    // sempre executado
}
```

Semântica:

- **try**: bloco monitorado
- **catch**: captura exceção, bind variável **error**
- **finally**: executa sempre (sucesso ou erro)
- Pelo menos **catch** ou **finally** deve estar presente

Definição 5.7 (Throw Statement). Sintaxe: **throw expression**;

Lança uma exceção que propaga pela call stack até ser capturada por **catch**.

5.4 Return Statement

Definição 5.8 (Return). Sintaxe: **return [expression]**;

Comportamento:

- Sai imediatamente da função
- Retorna valor especificado (ou **null** se omitido)
- Válido apenas dentro de funções ou threads

6 Declarações

6.1 Funções

Definição 6.1 (Function Declaration). **Sintaxe Padrão:**

```
function name(param1 [: type1], param2 [: type2], ...)
  [: returnType] {
    // body
    return value;
  }
```

Propriedades:

- Hoisted (pode ser chamada antes da declaração)
- Cria closure sobre escopo circundante
- Parâmetros podem ter valores default: `param = defaultValue`
- Suporta rest parameters: `...rest`

Definição 6.2 (Arrow Functions). **Sintaxe:**

```
const name = (params) => expression;
const name = (params) => { statements };
```

Diferenças de function:

- Não são hoisted
- Sintaxe mais concisa
- Return implícito se corpo é expressão única

Definição 6.3 (Async Functions). **Sintaxe:**

```
async function name(params) {
  let result = await asyncOperation();
  return result;
}
```

Comportamento:

- Pode usar `await` para esperar promises
- Retorna implicitamente uma promise
- Execução pode ser pausada em `await`

Definição 6.4 (Thread Functions). **Sintaxe:**

```
thread function name(params) {
  // executa em thread separada
  return result;
}
```

Invocação:

```
let future = thread name(args);
```

Comportamento:

- Executa em thread de OS separada
- Retorna handle para recuperar resultado
- Isolamento de contexto

6.2 Classes

Definição 6.5 (Class Declaration). **Sintaxe Básica:**

```
class ClassName {
    // Propriedades de instância
    property = defaultValue;

    // Construtor
    constructor(params) {
        this.property = value;
    }

    // Métodos de instância
    method() {
        return this.property;
    }

    // Métodos estáticos
    static staticMethod() {
        return value;
    }
}
```

Definição 6.6 (Herança). **Sintaxe:**

```
class Child extends Parent {
    constructor(params) {
        super(parentParams);
        // inicialização adicional
    }

    method() {
        super.method(); // chama método do pai
        // implementação adicional
    }
}
```

Propriedades:

- Suporta herança simples (um único pai)
- `super` referencia a classe pai
- `this` referencia a instância atual

Definição 6.7 (Modificadores de Acesso). • **public**: Acessível de qualquer lugar (default)

- **private**: Acessível apenas dentro da classe
- **protected**: Acessível na classe e subclasses

Aplicação:

```
class Example {
    public x = 1;      // acessível externamente
    private y = 2;     // apenas interno
    protected z = 3;  // classe e filhas
}
```

Definição 6.8 (Membros Estáticos). Membros declarados com `static` pertencem à classe, não às instâncias.

Exemplo:

```
class Counter {
    static count = 0;

    static increment() {
        Counter.count++;
    }
}

// Uso
Counter.increment();
console.log(Counter.count); // 1
```

7 Sistema de Módulos

7.1 Import Statement

Definição 7.1 (Importações Nomeadas). Sintaxe: `import { name1, name2, ... } from "module";`

Importa exports nomeados de um módulo.

Definição 7.2 (Importação Namespace). Sintaxe: `import * as namespace from "module";`
Importa todos os exports sob um namespace.

Uso: `namespace.function()`

Definição 7.3 (Importação Side-Effect). Sintaxe: `import "module";`
Executa o módulo por efeitos colaterais, sem importar valores.

Definição 7.4 (Importação por Identificador). Sintaxe: `import moduleName;`
 Importa módulo sem quotes (syntax moderna).

Acesso com namespace operator:

```
import ipe;
ipe::init();
ipe::config::value;
```

7.2 Export Statement

Definição 7.5 (Exports). **Export Direto:**

```
export function name() { }
export const VALUE = 42;
export class ClassName { }
```

Re-export:

```
export { name1, name2 } from "other_module";
export * from "other_module";
```

7.3 Native Directives

Definição 7.6 (Carregamento de Módulos Nativos). Sintaxe: `#<module_name>`
 Carrega módulo nativo (implementado em C/Rust) no escopo global.

Módulos Disponíveis:

- `#io` - Operações de I/O
- `#crypto` - Criptografia
- `#http` - Cliente/servidor HTTP
- `#math` - Funções matemáticas
- `#tcp`, `#udp` - Networking
- `#time` - Data e hora
- Outros módulos nativos

Exemplo:

```
#io

let content = io_read_file("data.txt");
io_write_file("output.txt", content);
```

8 Modelo de Execução

8.1 Arquitetura de Execução Híbrida

Definição 8.1 (Três Modos de Execução). Dryad suporta três estratégias de execução:

1. Tree-Walking Interpreter:

- Executa AST diretamente

- Modo padrão
- Otimizado para desenvolvimento e debugging
- Mais lento, mas mais flexível

2. Bytecode VM:

- Compila AST para bytecode stack-based
- Habilitado com `set_compile_mode(true)`
- VM executa OpCodes sequencialmente
- Performance intermediária

3. AOT Compiler (Experimental):

- Compila bytecode/IR para código nativo
- Gera executáveis nativos (ARM64, x86_64)
- Suporta formatos ELF e PE
- Máxima performance

8.2 Pipeline de Compilação

Definição 8.2 (Fluxo de Compilação). O processo de compilação segue:

$$\text{Source Code} \xrightarrow{\text{Lexer}} \text{Tokens} \xrightarrow{\text{Parser}} \text{AST}$$

$$\text{AST} \xrightarrow{\text{Interpreter}} \text{Result}$$

ou

$$\text{AST} \xrightarrow{\text{Bytecode Compiler}} \text{OpCodes} \xrightarrow{\text{VM}} \text{Result}$$

ou

$$\text{OpCodes} \xrightarrow{\text{IR Converter}} \text{IR} \xrightarrow{\text{Backend}} \text{Assembly} \xrightarrow{\text{Generator}} \text{Native Binary}$$

8.3 Escopo e Resolução de Variáveis

Definição 8.3 (Cadeia de Escopos). A hierarquia de escopos em Dryad:

Global Scope

Module Scope

Function Scope

Block Scope

Block Scope

...

...

Class Scope

Method Scope

...

Algoritmo de Lookup:

1. Verifica escopo atual
2. Se não encontrado, verifica escopo pai
3. Continua até escopo global
4. Se não encontrado em nenhum escopo, lança **ReferenceError**

Propriedade 8.1 (Regras de Binding). • **let** e **const**: block-scoped

- **function**: hoisted para escopo de função/módulo
- Variáveis globais: acessíveis de qualquer lugar
- Closures: funções capturam variáveis do escopo de definição

8.4 Execução de Funções

Definição 8.4 (Chamada de Função). Ao chamar uma função:

1. Cria novo escopo de função
2. Faz binding de parâmetros com argumentos
3. Inicializa variáveis declaradas no corpo
4. Executa statements em ordem
5. Retorna valor (explícito via **return** ou **null** implícito)
6. Destrói escopo de função

Definição 8.5 (Closures). Funções capturam variáveis do escopo de definição.

Propriedades:

- Variáveis capturadas permanecem vivas enquanto função existir
- Modificações em variáveis capturadas são visíveis
- Múltiplas closures podem compartilhar mesmas variáveis

Exemplo Conceitual:

```
function makeCounter() {  
  let count = 0; // capturada por closure  
  return function() {  
    return ++count; // modifica variável capturada  
  };  
}
```


8.5 Instanciação de Classes

Definição 8.6 (Processo `new ClassName()`). Ao executar `new ClassName(args)`:

1. Cria nova instância de objeto
2. Se `constructor` existe:
 - (a) Chama constructor com `this` bound à instância
 - (b) Executa corpo do constructor
3. Retorna a instância

Definição 8.7 (Execução de Métodos). Ao chamar `obj.method(args)`:

1. Cria escopo de função
2. Faz binding de `this` ao objeto
3. Faz binding de parâmetros com argumentos
4. Executa corpo do método
5. Retorna resultado

8.6 Representação Interna de Valores

Definição 8.8 (Value Enum (Conceitual)). Valores em runtime são representados por variante enum:

- `Number(f64)` - IEEE 754 floating-point
- `String(String)` - UTF-8 string
- `Bool(bool)` - booleano
- `Null` - valor nulo
- `Array(Vec<Value>)` - array dinâmico
- `Object(HashMap<String, Value>)` - mapa chave-valor
- `Function(...)` - referência a função
- `Class(...)` - definição ou instância de classe
- `Thread(...)` - handle de thread

8.7 Gerenciamento de Memória

Propriedade 8.2 (Garbage Collection). • **Estratégia:** Reference counting (via Rust)

- **Automático:** Sem intervenção do programador
- **Stack:** Valores primitivos permanecem na pilha
- **Heap:** Tipos complexos alocados no heap
- **Cleanup:** Valores liberados quando sem referências
- **Transparência:** Usuário não gerencia memória manualmente

9 Sistema de Erros

9.1 Categorias de Erros

Definição 9.1 (DryadError). Todos os erros em Dryad são instâncias de `DryadError` com:

- **Código:** Identificador numérico único
- **Categoria:** Tipo de erro
- **Mensagem:** Descrição legível
- **Localização:** Arquivo, linha, coluna
- **Stack Trace:** Pilha de chamadas
- **Contexto de Debug:** Sugestões, variáveis locais, documentação

Definição 9.2 (Categorias de Erro). 1. **Lexer** (1000-1999):

- Caractere inesperado
- String não fechada
- Número inválido
- Escape sequence inválida

2. **Parser** (2000-2999):

- Token inesperado
- Estrutura sintática inválida
- Parênteses não balanceados
- Statement esperado

3. **Runtime** (3000-3999):

- Variável indefinida
- Tipo incompatível
- Divisão por zero
- Stack overflow
- Propriedade não encontrada

4. **Type** (4000-4999):

- Incompatibilidade de tipos
- Número incorreto de argumentos

5. **I/O** (5000-5999):

- Arquivo não encontrado
- Permissão negada
- Operação de I/O falhou

6. **Module** (6000-6999):

- Módulo não encontrado
- Módulo nativo desconhecido
- Erro de resolução

7. **Syntax** (7000-7999):

- Erro de sintaxe geral

8. **Warning** (8000-8999):

- Avisos não-fatais
- Severidade: Low, Medium, High

9. **System** (9000+):

- Erros de sistema

9.2 Propagação de Exceções

Propriedade 9.1 (Propagação). Exceções não capturadas propagam pela call stack:

1. Exceção lançada em função profunda
2. Se não há `try-catch` local, propaga para chamador
3. Continua até encontrar `catch` ou chegar ao topo
4. Se não capturada, termina programa com erro

Definição 9.3 (Stack Trace). Cada exceção carrega stack trace com:

- Nome da função
- Localização (arquivo:linha:coluna)
- Contexto adicional

Ordenado do mais profundo ao mais superficial.

10 Modelo de Concorrência

10.1 Async/Await

Definição 10.1 (Funções Assíncronas). Funções declaradas com `async` podem usar `await`.
Comportamento:

- `async function` retorna promise implicitamente
- `await expr` suspende execução até promise resolver
- Permite código assíncrono com sintaxe síncrona

Modelo Conceitual:

```
async function process() {  
  let data = await fetchData(); // pausa aqui  
  return processData(data);      // continua quando resolve  
}
```

10.2 Threads Nativos

Definição 10.2 (Thread Functions). Funções declaradas com `thread function` podem executar em threads de OS.

Propriedades:

- Executa em thread separada do SO
- Isolamento de contexto
- Comunicação via mutexes ou retorno
- Handle retornado permite recuperar resultado

Exemplo Conceitual:

```
thread function backgroundTask(id) {  
  // executa em paralelo  
  return result;  
}  
  
let handle1 = thread backgroundTask(1);  
let handle2 = thread backgroundTask(2);  
// ambos executam simultaneamente
```

10.3 Sincronização com Mutex

Definição 10.3 (Mutex). Primitiva de sincronização para proteger recursos compartilhados.

API:

- `mutex()`: cria novo mutex

- `mu.lock()`: adquire lock (bloqueia se ocupado)
- `mu.unlock()`: libera lock

Uso:

```
let mu = mutex();
let sharedData = 0;

thread function increment() {
    mu.lock();
    sharedData++; // seção crítica
    mu.unlock();
}
```

11 Foreign Function Interface (FFI)

11.1 Sistema de Módulos Nativos

Definição 11.1 (Native Modules). Módulos implementados em C/Rust e expostos para Dryad.

Características:

- Carregados via native directives (`#<module>`)
- Funções tornam-se disponíveis no escopo global
- Tipagem dinâmica na interface
- Convenção de chamada C

11.2 Mapeamento de Tipos

Definição 11.2 (Type Mapping Dryad - C).

Dryad Type	C Type	Notas
number	double (f64)	IEEE 754
string	const char*	UTF-8 null-terminated
bool	bool / uint8_t	Booleano C
null	NULL / nullptr	Ponteiro nulo
array	void*	Ponteiro opaco
object	void*	Ponteiro opaco

Limitações:

- Tipos complexos passados como ponteiros opacos
- Apenas primitivos têm conversão direta
- Responsabilidade de conversão é da implementação nativa

11.3 Convenção de Chamada

Propriedade 11.1 (FFI Call Convention). Chamadas FFI seguem convenção C:

- Parâmetros passados left-to-right
- Stack-based (x86/x64)
- Valores de retorno em registrador **rax** (x86-64)
- Caller limpa stack (cdecl)

12 Arquitetura de Runtime Mínimo com Intrinsics

12.1 Filosofia do Self-Hosting

Definição 12.1 (Runtime Mínimo). Dryad adota uma filosofia de **self-hosting** onde:

- Runtime C++ fornece apenas **syscalls primitivas** (~50 intrinsics)
- Standard library implementada **100% em Dryad**
- Zero necessidade de escrever bindings manualmente
- Bibliotecas de alto nível (HTTP, JSON, crypto) escritas em Dryad puro

Propriedade 12.1 (Vantagens Arquiteturais). Esta abordagem, utilizada por linguagens modernas como Go, Zig e Rust, oferece:

- **Manutenção Única:** Não duplicar código em C++ e Dryad
- **Extensibilidade:** Usuários implementam bibliotecas em Dryad
- **Testabilidade:** Mock implementations sem tocar código nativo
- **Debuggability:** Todo código visível e inspecionável
- **Portabilidade:** Apenas syscalls precisam ser adaptadas por OS

12.2 Sistema de Intrinsics

Definição 12.2 (Intrinsic Function). Uma **intrinsic** é uma função especial reconhecida pelo compilador que:

1. Não gera instrução normal de chamada (**CALL**)
2. Compila diretamente para instrução assembly/opcode especializado
3. Tem custo de execução próximo a zero
4. Mapeia diretamente para código C++ do runtime

Definição 12.3 (Sintaxe de Declaração). Intrinsics são declaradas com decorator **@intrinsic**:

```
// Declaração de syscall intrinsic
@intrinsic("syscall.open")
extern function __open(path: string, flags: i32): i32;

@intrinsic("syscall.read")
extern function __read(fd: i32, buf: ptr<u8>, len: usize): isize;

@intrinsic("syscall.write")
extern function __write(fd: i32, buf: ptr<u8>, len: usize): isize;

@intrinsic("syscall.close")
extern function __close(fd: i32): void;
```

12.3 Pipeline de Compilação com Intrinsics

Definição 12.4 (Processo de Compilação). Durante compilação, ao encontrar chamada de função com `@intrinsic`:

1. **Parser**: Reconhece decorator e marca função como intrinsic
2. **Type Checker**: Valida assinatura contra definição do runtime
3. **Code Generator**: Emite opcode especial ao invés de `CALL`
4. **VM/AOT**: Executa código C++ diretamente

Exemplo de bytecode gerado:

```
// Dryad source
let fd = __open("file.txt", O_RDONLY);

// Bytecode emitido
LOAD_CONST "file.txt"
LOAD_CONST 0           // O_RDONLY
INTRINSIC_SYSCALL 1     // ID 1 = syscall.open
STORE_LOCAL fd
```

12.4 Implementação no Runtime C++

Definição 12.5 (Tabela de Syscalls). Runtime mantém tabela de IDs de syscalls:

```
// C++ implementation
enum class SyscallID : uint16_t {
    OPEN = 1,
    READ = 2,
    WRITE = 3,
    CLOSE = 4,
    SOCKET = 5,
    CONNECT = 6,
    // ... ~50 syscalls total
};
```

Definição 12.6 (Dispatcher de Intrinsics). VM executa intrinsics via dispatch table:

```
void VM::execute_intrinsic(SyscallID id) {
    switch (id) {
        case SyscallID::OPEN: {
            auto flags = pop_stack().as_int();
            auto path = pop_stack().as_string();
            int fd = ::open(path.c_str(), flags);
            push_stack(Value::from_int(fd));
            break;
        }
        case SyscallID::READ: {
            auto len = pop_stack().as_int();
            auto buf = pop_stack().as_buffer();
            auto fd = pop_stack().as_int();
            ssize_t n = ::read(fd, buf.data(), len);
            push_stack(Value::from_int(n));
            break;
        }
        // ... outros syscalls
    }
}
```

12.5 Standard Library em Dryad Puro

Definição 12.7 (Camadas de Abstração). Standard library organizada em camadas:

High-Level APIs (100% Dryad)

- http.dryad (HTTP client/server)
- json.dryad (parser/serializer)
- crypto.dryad (algorithms)

↓ usa

Core I/O (Dryad + Intrinsics)

- io.dryad (FileSystem, Buffer)
- net.dryad (Socket, TCP/UDP)

↓ chama

Runtime Intrinsics (~50 syscalls)

- syscall.open/read/write/close
- syscall.socket/connect/send/recv
- memory.alloc/free

Definição 12.8 (Exemplo: FileSystem em Dryad). Implementação completa de I/O em Dryad puro:


```

// @std/io.dryad
@intrinsic("syscall.open")
extern function __open(path: string, flags: i32): i32;

@intrinsic("syscall.read")
extern function __read(fd: i32, buf: ptr<u8>, len: usize): isize;

@intrinsic("syscall.close")
extern function __close(fd: i32): void;

const O_RDONLY = 0;
const O_WRONLY = 1;
const O_CREAT = 64;

export class File {
  private fd: number;

  constructor(path: string, mode: string = "r") {
    let flags = mode == "r" ? O_RDONLY : (O_WRONLY | O_CREAT);
    this.fd = __open(path, flags);
    if (this.fd < 0) {
      throw new Error("Failed to open: " + path);
    }
  }

  read(): string {
    let buffer = Buffer.allocate(4096);
    let n = __read(this.fd, buffer.ptr, 4096);
    if (n < 0) throw new Error("Read failed");
    return buffer.slice(0, n).toString();
  }

  close(): void {
    __close(this.fd);
  }
}

export function readFile(path: string): string {
  let f = new File(path, "r");
  let content = f.read();
  f.close();
  return content;
}

```

12.6 Virtual File System (VFS)

Definição 12.9 (Arquitetura VFS). Sistema de arquivos virtual com backends plugáveis:

```
interface FileSystemBackend {
```

```

    open(path: string, mode: string): FileHandle;
    read(handle: FileHandle, size: number): Buffer;
    write(handle: FileHandle, data: Buffer): number;
    close(handle: FileHandle): void;
}

// Backend nativo (usa syscalls)
class NativeBackend implements FileSystemBackend {
    open(path: string, mode: string): FileHandle {
        let flags = this.parseMode(mode);
        let fd = __open(path, flags);
        return new NativeFileHandle(fd);
    }
    // ... implementação com syscalls
}

// Backend in-memory (100% Dryad, zero syscalls!)
class MemoryBackend implements FileSystemBackend {
    private storage: Map<string, Buffer> = new Map();

    open(path: string, mode: string): FileHandle {
        if (!this.storage.has(path) && mode == "w") {
            this.storage.set(path, Buffer.allocate(0));
        }
        return new MemoryFileHandle(path, this.storage);
    }
    // ... sem syscalls, apenas Map em memória
}

```

Propriedade 12.2 (Benefícios do VFS). • **Testabilidade:** Testes usam MemoryBackend sem tocar disco

- **Extensibilidade:** Usuários criam backends customizados (HTTP, S3, etc)
- **Segurança:** Backends podem implementar sandboxing
- **Composabilidade:** Backends podem ser combinados (cache, logging)

12.7 Networking e Async I/O

Definição 12.10 (Event Loop em Dryad). Para suportar operações assíncronas, event loop implementado em Dryad:

```

// @std/async/event_loop.dryad
@intrinsic("syscall.epoll_create")
extern function __epoll_create(): i32;

@intrinsic("syscall.epoll_wait")
extern function __epoll_wait(epfd: i32, timeout: i32): Array<i32>;

```

```

@intrinsic("syscall.epoll_ctl")
extern function __epoll_ctl(epfd: i32, op: i32, fd: i32): void;

class EventLoop {
  private epoll_fd: number;
  private tasks: Map<number, Task> = new Map();

  constructor() {
    this.epoll_fd = __epoll_create();
  }

  run(): void {
    while (this.tasks.size > 0) {
      // Multiplexação de I/O não-bloqueante
      let ready_fds = __epoll_wait(this.epoll_fd, 100);

      for (let fd of ready_fds) {
        let task = this.tasks.get(fd);
        if (task) {
          task.resume(); // Acorda coroutine
        }
      }
    }
  }

  register(fd: number, task: Task): void {
    __epoll_ctl(this.epoll_fd, EPOLL_CTL_ADD, fd);
    this.tasks.set(fd, task);
  }
}

```

Definição 12.11 (HTTP Client em Dryad Puro). Cliente HTTP implementado sem código C++:

```

// @std/http.dryad
import { Socket } from "@std/net";
import { EventLoop } from "@std/async";

export class HttpClient {
  async get(url: string): Response {
    // Parse URL (puro Dryad)
    let parsed = this.parseUrl(url);

    // Socket connection (syscall socket/connect)
    let socket = new Socket(parsed.host, parsed.port);
    await socket.connect();

    // Build HTTP request (puro Dryad)
    let request = this.buildRequest("GET", parsed.path);
  }
}

```

```

    // Send (syscall write)
    await socket.write(request);

    // Receive (syscall read via event loop)
    let response = await socket.readAll();

    // Parse response (puro Dryad)
    return this.parseResponse(response);
}

private parseUrl(url: string): UrlInfo { ... }
private buildRequest(method: string, path: string): string { ... }
private parseResponse(data: string): Response { ... }
}

```

12.8 Gerenciamento de Memória com Buffer

Definição 12.12 (Classe Buffer). Abstração type-safe para manipulação de bytes:

```

// @std/buffer.dryad
@intrinsic("memory.allocate")
extern function __alloc(size: usize): ptr<u8>;

@intrinsic("memory.free")
extern function __free(ptr: ptr<u8>): void;

export class Buffer {
  private data: ptr<u8>;
  private _size: number;

  static allocate(size: number): Buffer {
    let ptr = __alloc(size);
    return new Buffer(ptr, size);
  }

  static fromString(s: string): Buffer {
    let size = s.length;
    let buf = Buffer.allocate(size);
    // Copia bytes de string para buffer
    for (let i = 0; i < size; i++) {
      buf.set(i, s.charCodeAt(i));
    }
    return buf;
  }

  toString(): string {
    // Converte bytes para string
    let chars = [];

```

```

    for (let i = 0; i < this._size; i++) {
        chars.push(String.fromCharCode(this.get(i)));
    }
    return chars.join("");
}

get ptr(): ptr<u8> { return this.data; }
get size(): number { return this._size; }
}

```

Propriedade 12.3 (Segurança de Memória). Buffer encapsula ponteiros brutos e fornece:

- **Bounds checking:** Verificação de limites em runtime
- **Automatic cleanup:** GC libera memória automaticamente
- **Type safety:** Conversões explícitas entre Buffer e tipos primitivos
- **Zero-copy:** Passa ponteiro direto para syscalls

12.9 Conjunto Mínimo de Syscalls

Definição 12.13 (Syscalls Essenciais). Runtime C++ implementa aproximadamente 50 syscalls:

File I/O:

- open, read, write, close
- lseek, stat, unlink, mkdir

Network:

- socket, connect, bind, listen
- accept, send, recv, shutdown

Memory:

- malloc, free, realloc
- memcpy, memset

Async I/O:

- epoll_create, epoll_ctl, epoll_wait (Linux)
- kqueue, kevent (macOS/BSD)
- select (fallback portátil)

Process/Thread:

- fork, exec, wait
- pthread_create, pthread_join

Time:

- gettimeofday, clock_gettime
- sleep, nanosleep

12.10 Comparação com Sistema Anterior

Aspecto	Sistema Antigo	Sistema Novo
Stdlib	Wrappers C++ manuais	100% Dryad
Manutenção	Duplicada (C++ + Dryad)	Única (apenas Dryad)
Extensibilidade	Requer código C++	Bibliotecas em Dryad
Testabilidade	Difícil (syscalls reais)	Backends mock
Runtime Size	Grande (muitos bindings)	Mínimo (50 syscalls)
Performance	Overhead de wrapper	Intrinsics diretas
Portabilidade	Adaptar cada binding	Adaptar syscalls
Debuggability	Parte em C++ opaco	Tudo visível em Dryad

13 Bytecode e IR

13.1 Bytecode VM

Definição 13.1 (OpCodes). O bytecode de Dryad consiste em OpCodes stack-based.

Categorias de OpCodes:

- **Stack:** Push, Pop, Dup
- **Aritméticos:** Add, Sub, Mul, Div, Mod, Pow
- **Lógicos:** And, Or, Not
- **Bitwise:** BitwiseAnd, BitwiseOr, BitwiseXor, Shift
- **Comparação:** Equal, NotEqual, Less, Greater, LessEqual, GreaterEqual
- **Controle:** Jump, JumpIfFalse, JumpIfTrue, Call, Return
- **Variáveis:** GetLocal, SetLocal, GetGlobal, SetGlobal
- **Funções:** DefineFunction, CallFunction
- **Objetos:** GetProperty, SetProperty, NewObject, NewArray
- **Classes:** DefineClass, NewInstance, CallMethod

Definição 13.2 (Stack-Based Execution). A VM mantém:

- **Stack:** Pilha de valores para operandos e resultados
- **Instruction Pointer (IP):** Posição atual no bytecode
- **Call Stack:** Pilha de frames de função
- **Globals:** Ambiente de variáveis globais

Ciclo de Execução:

1. Fetch: Lê OpCode no IP
2. Decode: Identifica operação e operandos

3. Execute: Realiza operação (manipula stack)
4. Advance: Incrementa IP
5. Repeat até Return ou fim

13.2 Intermediate Representation (IR)

Definição 13.3 (IR Dryad). Representação intermediária usada pelo compilador AOT.

Características:

- Mais abstrata que assembly, mais concreta que AST
- Register-based (ao contrário do bytecode stack-based)
- Independente de arquitetura
- Permite otimizações antes de geração de código

Estrutura:

- IrModule: Unidade de compilação completa
- IrFunction: Definição de função com parâmetros e corpo
- IrInstruction: Instruções IR (Load, Store, Add, Call, etc.)
- IrValue: Valores (constantes, variáveis, registradores)

14 Compilação AOT (Ahead-of-Time)

14.1 Pipeline AOT

Definição 14.1 (Fases do Compilador AOT). 1. **Bytecode $\rightarrow$ IR Conversion:**

- Converte OpCodes stack-based para IR register-based
- Análise de fluxo de controle
- Construção de CFG (Control Flow Graph)

2. **IR Optimization:**

- Constant folding
- Dead code elimination
- Register allocation

3. **Backend Code Generation:**

- Seleção de instruções para arquitetura alvo
- Alocação de registradores físicos
- Geração de assembly (ARM64 ou x86_64)

4. **Object File Generation:**

- Geração de formato executável (ELF ou PE)
- Linking com runtime libraries
- Produção de binário nativo

14.2 Backends Suportados

Definição 14.2 (Arquiteturas). • **ARM64:**

- Arquitetura RISC de 64 bits
- Registradores: x0-x30, sp, pc
- Instrução fixa de 32 bits

• **x86_64:**

- Arquitetura CISC de 64 bits
- Registradores: rax, rbx, rcx, rdx, rsi, rdi, rsp, rbp, r8-r15
- Comprimento de instrução variável

14.3 Formatos de Objeto

Definição 14.3 (Geradores de Formato). • **ELF Generator:**

- Executable and Linkable Format
- Usado em Linux/Unix
- Seções: .text, .data, .bss, .rodata

• **PE Generator:**

- Portable Executable
- Usado em Windows
- Seções: .text, .data, .rdata

15 Padrões de Programação

15.1 Paradigma Orientado a Objetos

Definição 15.1 (OOP em Dryad). Dryad suporta OOP completo:

- **Encapsulamento:** Modificadores de acesso
- **Herança:** Via `extends`
- **Polimorfismo:** Métodos podem ser sobrescritos
- **Abstração:** Classes e interfaces

15.2 Paradigma Funcional

Definição 15.2 (Functional Programming em Dryad). Características funcionais:

- **First-class functions:** Funções como valores
- **Higher-order functions:** Funções que recebem/retornam funções
- **Closures:** Captura de escopo
- **Arrow functions:** Sintaxe concisa
- **Imutabilidade parcial:** `const` para variáveis

15.3 Paradigma Procedural

Definição 15.3 (Procedural em Dryad). Suporte a programação procedural tradicional:

- Funções top-level
- Variáveis globais e locais
- Controle de fluxo estruturado
- Sem necessidade de classes

16 Especificações Formais

16.1 Gramática Formal (BNF Simplificada)

Definição 16.1 (Gramática de Dryad (Subset)). `<program> ::= <statement>*`

```

<statement> ::= <var-decl>
               | <const-decl>
               | <function-decl>
               | <class-decl>
               | <if-stmt>
               | <while-stmt>
               | <for-stmt>
               | <return-stmt>
               | <throw-stmt>
               | <try-catch-stmt>
               | <expression-stmt>
               | <import-stmt>
               | <export-stmt>
               | <block-stmt>

```

```

<var-decl> ::= "let" <identifier> [":" <type>] ["=" <expression>] ";"

```

```

<const-decl> ::= "const" <identifier> [":" <type>] "=" <expression> ";"

```

```

<function-decl> ::= ["async"] "function" <identifier>
                  "(" <parameter-list> ")" [":" <type>]
                  "{" <statement>* "}"

<class-decl> ::= "class" <identifier> ["extends" <identifier>]
               "{" <class-member>* "}"

<expression> ::= <literal>
                | <identifier>
                | <binary-expr>
                | <unary-expr>
                | <call-expr>
                | <member-expr>
                | <lambda-expr>
                | <array-literal>
                | <object-literal>
                | <match-expr>

<binary-expr> ::= <expression> <binary-op> <expression>

<unary-expr> ::= <unary-op> <expression>

<call-expr> ::= <expression> "(" <argument-list> ")"

<member-expr> ::= <expression> "." <identifier>
                | <expression> "[" <expression> "]"
                | <expression> "::" <identifier>

<lambda-expr> ::= "(" <parameter-list> ")" "=" <expression>
                | "(" <parameter-list> ")" "=" "{" <statement>* "}"

<type> ::= "number" | "string" | "bool" | "any"
         | <type> "[]"
         | "(" <type-list> ")"
         | "fn" "(" <type-list> ")" "->" <type>

```

16.2 Semântica Operacional (Informal)

Definição 16.2 (Regras de Avaliação). **Expressões Aritméticas:**

- $n_1 + n_2 = n_1 + n_2$ (se ambos números)
- $s_1 + s_2 = \text{concat}(s_1, s_2)$ (se algum é string)
- $n_1/n_2 = n_1/n_2$ se $n_2 \neq 0$, senão erro

Expressões Lógicas:

- $a \&\& b = \begin{cases} a & \text{if } \text{isFalsy}(a) \\ b & \text{otherwise} \end{cases}$

- $a||b = \begin{cases} a & \text{if } isTruthy(a) \\ b & \text{otherwise} \end{cases}$

Resolução de Variáveis:

- $x_\rho = \rho(x)$ se $x \in dom(\rho)$
- $x_\rho = x_{\rho'}$ se $x \notin dom(\rho)$ e ρ' é escopo pai
- Erro se x não encontrado em nenhum escopo

17 Limitações e Considerações

17.1 Limitações Conhecidas

- **Performance:** Interpretador é mais lento que linguagens compiladas
- **Type Safety:** Sistema de tipos dinâmico não previne erros de tipo em compile-time
- **FFI:** Limitações ao passar tipos complexos para C
- **Concorrência:** Interpreter base é single-threaded (threads são isoladas)
- **Módulos:** Sistema de módulos sem versionamento
- **AOT:** Suporte experimental, nem todas features suportadas

17.2 Características de Performance

- Propriedade 17.1** (Trade-offs). • **Adequado para:** Scripting, automação, prototipagem, aplicações pequenas/médias
- **Inadequado para:** Computação intensiva, sistemas real-time, aplicações críticas de performance
 - **Bottlenecks:** Call stacks profundas, operações em arrays grandes, GC frequente

18 Conclusão

18.1 Resumo

Dryad é uma linguagem de programação multi-paradigma moderna que combina:

- Sintaxe familiar (JavaScript/TypeScript-like)
- Tipagem dinâmica com anotações opcionais
- Modelo de execução híbrido (interpreter/bytecode/AOT)
- Sistema de módulos robusto com FFI
- Concorrência nativa (async/await e threads)
- Sistema de erros estruturado e informativo

18.2 Filosofia de Design

A linguagem prioriza:

1. **Clareza e Legibilidade:** Código deve ser auto-explicativo
2. **Flexibilidade:** Suporte a múltiplos paradigmas
3. **Developer Experience:** Erros informativos, sintaxe consistente
4. **Gradualidade:** Pode começar simples e adicionar complexidade quando necessário

18.3 Direções Futuras

Possíveis evoluções da linguagem:

- Maturação do compilador AOT
- Otimizações de performance no bytecode VM
- Expansão de módulos nativos padrão
- Sistema de tipos gradual mais robusto
- Ferramentas de análise estática
- Package manager integrado

Apêndice A: Tabela de Códigos de Erro

Faixa	Categoria
1000-1999	Lexer Errors
2000-2999	Parser Errors
3000-3999	Runtime Errors
4000-4999	Type Errors
5000-5999	I/O Errors
6000-6999	Module Errors
7000-7999	Syntax Errors
8000-8999	Warnings
9000+	System Errors

Apêndice B: Palavras Reservadas (Completo)

- | | | |
|------------|-----------|------------|
| • async | • else | • function |
| • await | • export | • if |
| • break | • extends | • import |
| • catch | • false | • in |
| • class | • finally | • let |
| • const | • fn | • match |
| • continue | • for | • mutex |
| • do | • from | • new |

- | | | |
|-------------|----------|---------|
| • null | • static | • true |
| • private | • super | • try |
| • protected | • this | • use |
| • public | • thread | • while |
| • return | • throw | |

Referências

1. STANDARDIZATION_MANIFEST.md - Dryad Project Standardization
2. DRYAD_LANGUAGE_SPECIFICATION_FOR_LLM.md - Complete Language Specification
3. Dryad Error Catalog - Centralized Error Definitions
4. Dryad Source Code - Implementation Reference (Rust)